Nonhalting and PA-independence for a 209-state Turing machine

James Chang

Abstract

We give an explicit two-symbol Turing machine with 209 working states that searches for a contradiction in ordinary first-order Peano arithmetic. The machine runs forever on a blank tape, and Peano arithmetic proves neither its exact arithmetic halting sentence nor its negation. The construction connects a register-program simulation to a refutation compiler that works in every model of Peano arithmetic. The literal transition table, both unprovability directions, and blank-tape nonhalting are formalized in Lean 4.

1 The result

Busy Beaver problems ask how long a halting machine of a fixed size can run [1]. Explicit small machines whose behavior is independent of a specified theory also give concrete examples of incompleteness. Yedidia and Aaronson constructed such a machine for ZFC [2]. Here the theory is ordinary first-order Peano arithmetic with full induction, denoted by PA. This construction reduces the working-state count from the earlier 211-state construction for PA [3]. The register engine draws on the K230 construction [4], and its proof calculus uses ingredients from the PA238 development [5] and Reshetnikov's restricted implication proof [6].

Write M for the machine in Section 2. Let H be the sentence of arithmetic defined in Section 5, and put $N = \neg H$. Write $\mathrm{PA} \vdash A$ for the existence of a finite formal derivation of A from PA. The statement Con(PA), used inside arithmetic, is the syntactic consistency sentence for the standard provability predicate.

Theorem 1.1. *The machine M runs forever from the blank tape. Its arithmetic nonhalting sentence is true in $\mathbb{N}$, and*

$$\mathrm{PA} \nvdash H \qquad \textit{and} \qquad \mathrm{PA} \nvdash N.$$

Under the external assumption of PA consistency, the machine runs forever and $\mathrm{PA} \nvdash N$. The proof of $\mathrm{PA} \nvdash H$ uses the soundness of PA in the standard natural numbers. In ordinary classical mathematics that standard model also establishes consistency. Lean uses the standard-model theorems for Foundation's theory `Peano` [8]. The published statement has an explicit presentation of the ordinary PA proof calculus, with its equivalence to Foundation's presentation proved in the Solution.

2 The machine and its startup

The tape is infinite in both directions, with alphabet $\{0, 1\}$ and initially all cells zero. State 0 is the initial state. The working state labels are

$$Q = \{0, \ldots, 213\} \setminus \{50, 122, 150, 157, 205\}.$$

A row (q, b, w, D, q') reads b at state q, writes w, moves one cell in direction D, and enters q'. There is one row for every pair in $Q \times \{0, 1\}$. The state 215 terminates the run. The 418 rows are given in `data/pa209.tm`, with SHA-256 digest

`0610d978675320f0b4b1ef34619dcd3daa5912632d7037e200dfcc70954fea51`.

For example, the first row is $(0, 0, 1, R, 1)$ and the transition $(68, 1, 1, R, 215)$ enters the halt state. Lean checks closure of all row targets in $Q \cup \{215\}$ and the count of distinct working states.

The exact startup takes 58,210 tape steps. It reaches the frame for source site 25 at literal control 81, with the candidate register equal to 2 and every other logical register zero. The complete register file, in its physical order, is

$$(0, 0, 0, 0, 0, 0, 2, 0, 0, 1^{[26]}),$$

where $1^{[26]}$ denotes 26 further entries equal to 1. The prefix checks the full tape and head, including those spare entries. The definitions and proof are in `M6/Startup/Data.lean` and `M6/Startup.lean`.

3 Simulation of the register program

The 3,948-site physical source is obtained by flattening the binary tree in `M6/Program.lean`. Its instructions increment a register, decrement a register or branch on zero, transfer control to a structured loop boundary, or halt. The source crosswalk collapses the extra physical site and normalizes the control transfers to the 3,947-site command program. The checked clean frame corresponds to the zero command store at candidate 2 in that program.

A frame contains the source path, header and marker, and a register file in which a value v is represented by a block $0\,1^{v+1}$. The physical proof relabels the literal controls, sending the start and halt to 74 and 254 respectively. The relabeling commutes with each step, preserving the two tape lists. The resulting contracts are proved for every source site and for both outcomes of a conditional decrement. Each contract takes a frame to the complete frame of its source successor in a positive number of tape steps, or implements the source halt. The proof combines the finite control diagrams with the unary arithmetic and register-file lemmas. It checks source addresses, control destinations, and register effects separately.

The two right moves $(181, 1, 1, R, 13)$ and $(187, 0, 0, R, 75)$ supply the changed copy operations used by the sixth axiom head in Section 4. The second move reaches control 75 with the preceding bits 10. The following are its two preserving control paths, with offsets relative to the head on entry at 75.

$$\begin{array}{ll} 0 & 75@0,\ 143@{-}1,\ 149@{-}2,\ 113@{-}1,\ 146@0,\ 82@1, \\ 1 & 75@0,\ 139@{-}1,\ 171@0,\ 83@1. \end{array}$$

The resulting pair of providers $(82, 83)$ performs the composition whose first branch copies the register holding the second axiom argument into the arithmetic accumulator. The branch entering 13 uses the same copy operation. The four changed source operands are decrement, increment, decrement, and increment, replacing register 3 by register 5 within the two copies. The checked contracts verify the actual entry histories, branch exits, and intervening providers against the complete table. The source crosswalk and the source command specification check the resulting logical argument order.

Lemma 3.1 (Simulation)**.** *If the command program from the clean candidate-2 store does not halt, then the literal machine does not halt from the blank tape.*

Proof. Append the certified startup to the positive-length simulation of a source prefix. A source execution with arbitrarily long prefixes gives tape runs of arbitrarily large length from the same initial configuration. Determinism excludes a finite terminal run. The proof uses the all-site coverage theorem and the crosswalk to the command program. □

The model-valued version has an additional role. It describes the same tape operations by arithmetic relations and concatenates their coded traces in a model of PA. The certificate proof in Section 6 uses that version.

4 Proof search and soundness

4.1 Proof commands

Let $p(a,b) = \binom{a+b+1}{2} + b$ be Cantor pairing. Formula codes use $I(a,b) = 4p(a,b)$ for implication and $U(a,b) = 4p(a,b) + 2$ for universal quantification. The two atomic forms have residues 1 and 3 for membership and equality. Code 0 is truth, and code 1 decodes as $x_0 \in x_0$. The decoder interprets implication into code 1 as negation. The calculus uses modus ponens and generalization, and its axiom formulas are the exact constructors in `ActualAXCode.lean`. The eighth head requires a dominance condition on its fourth parameter. The separation formula at the gate has the two indicated freshness conditions on the third variable.

A store (t, w, d, h) has two visible formula codes t, h, a stack w encoded by iterated pairing, and a variable-label offset d. Write p_L, p_R for the total Cantor projections. The three commands are as follows.

Command	Effect
S	Halt if $t + h = 1$. Otherwise replace (t, w, h) by $(h, p_R(w), p_L(w))$.
$\mathsf{MP}(x)$	Put $y = d + x$. If $h = I(t, y)$, return $(y, p(0, w), y, 0)$. Otherwise return $(0, w, y, 0)$.
$\mathsf{AX}(c, x, y)$	Put $a = d + x$ and $b = h + y$. Push eight axiom heads, $U(d, h)$, and the saved top t. Set t to the guarded separation formula, or to truth when $c = a$ or $c = b$. Set $d = h = 0$.

The sixth head in this ordering is

$$\rho_6(a,b) = I\big(\ulcorner \exists x_a \, \forall x_a \, B \urcorner, b\big),$$

where B has code b and the quantifiers use the numeric constructors. The choice of arguments follows from the checked copy operations in Section 3. The other seven heads and the generalization head keep their stated parameter dependence.

4.2 Soundness and enumeration

Interpret $y \in x$ in a model of PA as the statement that bit y of x is set. The uniform axiom suppliers establish truth under this interpretation, including pointed separation and the guarded implicational schema. Modus ponens and generalization preserve validity. The relation $y \in x$ implies $y < x$, so code 1 is false under every assignment in this interpretation. Soundness for an actual finite derivation, followed by first-order completeness, gives a PA refutation if code 1 has a proof in the command calculus.

The clean store contains truth in the visible registers and in the initial stack tail. The soundness invariant requires derivations for the visible formulas and each entry in the relevant stack

prefix. An AX command supplies its heads with their derivations, an MP command keeps the derived consequent or truth, and a Shift moves derivations between the visible positions and stack. The only halt test is $t + h = 1$, which requires one visible formula to have code 1. The source scheduler preserves the invariant from the clean candidate-2 store. It reads candidate codes by repeated unpairing, consumes the command atoms in order, and proceeds to the next candidate after a word finishes. The decoder and each candidate's register computation terminate or take the designated halt branch.

Thus under Con(PA) every candidate computation continues past the halt test. Lemma 3.1 proves blank-tape nonhalting. This direction uses the external soundness theorem for actual finite derivations in the command calculus.

5 The exact arithmetic trace sentence

Encode a tape by natural numbers l, r. Their low bits are respectively the cell just to the left of the head and the scanned cell. Higher bits run away from the head. Missing high bits represent zero. For a right-moving row (q_0, b, w, R, q_1) use the arithmetic relation

$$q = q_0 \wedge q' = q_1 \wedge \exists u\,(r = 2u + b \wedge l' = 2l + w \wedge r' = u).$$

For a left-moving row use

$$\begin{aligned} q = q_0 \wedge q' = q_1 \wedge \exists u, v, e\,((e = 0 \vee e = 1) \wedge l = 2v + e \wedge r = 2u + b \\ \wedge\, l' = v \wedge r' = 2(2u + w) + e). \end{aligned}$$

Let $F(q, l, r, q', l', r')$ be the disjunction of these formulas over the 418 rows. These formulas use the published state labels. Writing sequence access in subscript notation, the halting sentence is

$$H := \exists n, Q_*, L_*, R_* \begin{bmatrix} Q_0 = L_0 = R_0 = 0 \wedge Q_n = 215 \\ \wedge\, \forall i < n\ F(Q_i, L_i, R_i, Q_{i+1}, L_{i+1}, R_{i+1}) \end{bmatrix}. \tag{1}$$

The three sequence variables range over arithmetic codes. Each access in (1) abbreviates the explicit arithmetic sequence relation `nthDef`. In the standard model these relations describe exactly the finite tape runs. In every model of PA, the exact formula is equivalent to the auxiliary formula used to compose coded trace segments. For this equivalence, induction on the coded step index excludes the six padding labels $50, 122, 150, 157, 205, 214$ from a trace starting at zero. `ExactHaltingFormula.lean` proves the row correspondence and the two trace implications.

6 From a coded PA refutation to halting

Fix an arbitrary model V of PA. All objects in this section are arithmetically coded in V. In particular, proof length, recursion fuel, and trace length may be nonstandard.

6.1 Refutation transport

The arithmetized transport starts with a proof code for $\bot$ in ordinary PA. It translates the sequent proof to the presentation with successor, and then interprets that presentation in the numeric calculus of membership and equality. The interpretation uses the finite ordinal domain with zero and ordinal successor, and treats equality by the ordinal isomorphism relation. The

axiom suppliers realize the arithmetic defining equations and the induction scheme by separation and ordinal induction. Logical inference is transported by arithmetic operations on proof codes. This produces an accepted certificate for code 1 uniformly in V. The formal transport is `FoundationNativeRefutationTransport.lean`. It supplies the internal implication from standard PA provability of $\bot$ to the existence of a certificate satisfying the exact refutation-checker graph.

6.2 Compiling an accepted certificate

A certificate is $p(f, i)$, with fuel f and a tree code i. A tree code pairs a rule tag with its payload. Payload tuples are iterated Cantor pairs ending in zero. Tags $0, 1$ are modus ponens and generalization. Tags 2 through 9 are the eight axiom heads, in order, and tag 10 is the guarded separation instance. Failed guards, malformed tuples and rejected premises reject the certificate. The checker returns $a + 1$ on accepting formula code a, and returns zero on rejection. Thus the refutation-checker graph tests output 2.

The command compiler uses exact finite words to store, retrieve, and discard formulas. In the following compositions, words execute from left to right. The identity

$$\mathsf{AX}(c, x, y)\,\mathsf{S}^{10}(t, w, d, h) = (U(d, h), w, 0, t)$$

follows by following the ten pushed items. The first ten Shift tests see the gate and heads with residues 0 or 2 modulo 4. The following words give the required storage operations.

$$\begin{aligned}\mathsf{POP} &= \mathsf{AX}(0,0,0)\,\mathsf{S}^{11},\\ \mathsf{PUSH} &= \mathsf{AX}(0,0,0)\,\mathsf{S}^{9}\,\mathsf{MP}(0),\\ \mathsf{DROP} &= \mathsf{MP}(1)\,\mathsf{AX}(1,0,0)\,\mathsf{S}^{10}\,\mathsf{MP}(0).\end{aligned}$$

On a stack $p(a, w)$, the first word preserves t and w and returns $h = a, d = 0$. The second returns $(0, p(t, w), 0, 0)$ because the generalization head cannot match the MP test. On $(t, w, 0, 0)$ the third returns $(0, w, 0, 0)$, since its two MP tests reject.

To select head j for $1 \le j \le 7$, clear the top and use

$$\mathsf{DROP}\,\mathsf{AX}(c, a, b)\,\mathsf{S}^{j+1}\,\mathsf{POP}^{9-j}.$$

For the sixth head, use $\mathsf{AX}(0, a, b)$ to obtain $\rho_6(a, b)$. For head eight with fourth parameter $u \le a$, first set the offset to u after clearing the top and use $\mathsf{AX}(c, a-u, b)$. Then use $\mathsf{S}^9\mathsf{POP}$. A nonzero offset u is set by $\mathsf{MP}(u)$ on a zero top and zero caches. The zero offset requires the empty word. The gate selector is $\mathsf{DROP}\,\mathsf{AX}(c, a, b)\,\mathsf{POP}^{10}$. These words leave the original stack tail and clear both caches.

Generalization of a with label i uses PUSH, sets offset i, then executes $\mathsf{S}\,\mathsf{AX}(i, 0, 0)\,\mathsf{S}^{10}$. For MP with premises a and $I(a, b)$, compile the implication, execute PUSH, compile a, and execute PUSH again. The suffix is

$$\mathsf{S}^2\,\mathsf{MP}(b)\,\mathsf{AX}(b, 0, 0)\,\mathsf{S}^{11}.$$

The second Shift halts if $a = 1$. Otherwise the MP succeeds, pushes its zero, and the remaining commands return b with the original stack tail and zero caches. Each of these calculations uses polynomial identities, total Cantor projections, and finite command equations, so it holds in V.

For a formal induction in V, let $A(f, B)$ be the coded vector of checker outputs for indices $0, \dots, B$ at fuel f, and let $W(f, B)$ be the vector of compiled words at the same indices. The fuel-zero vectors contain rejections and empty words respectively. A successor row reads the children

from the previous vectors, applies the rule just described, and appends one coded word as one entry. Let $\mathcal{R}(w, a)$ mean that, for all t, z, running w on $(t, z, 0, 0)$ halts or returns $(a, z, 0, 0)$. Full PA induction on f proves

$$\forall B, i, a \quad A(f, B)_i = a + 1 \implies \mathcal{R}(W(f, B)_i, a).$$

At zero fuel the antecedent is false. At successor fuel an accepted leaf uses its exact selector, a generalization uses the child hypothesis followed by its suffix, and an MP uses both child hypotheses followed by the MP suffix. Rejected rows cannot meet the antecedent. This induction concerns an arithmetic predicate on coded vectors and stores. It applies to all fuel values in V.

For a certificate accepting 1, prefix the resulting word with $\mathsf{AX}(0, 0, 0)$ and append a Shift. The prefix clears the two caches from any initial store. If compilation has not already halted, the last Shift sees $(t, h) = (1, 0)$ and halts. This gives a single word that halts from every store in V.

6.3 Reaching the word and composing the trace

The universal halting word begins with $\mathsf{AX}(0, 0, 0)$, so its first atom has tag at least 2. The word encoding gives a candidate index at least 3, after the actual startup at candidate 2. The scheduler's coded induction up to this index yields either an earlier halt or a run of the universal word. This uses total-or-halt for each earlier candidate and permits its resulting store to be arbitrary.

The source-to-tape correspondence replaces every step in the coded source trace by a positive-length coded tape block. It starts from the full clean frame, concatenates these blocks, and prepends the certified blank startup. The resulting coded trace ends at state 215. The row and trace correspondence in Section 5 puts it in the exact formula H. Because V was arbitrary, first-order completeness gives the internal implication

$$\mathrm{PA} \vdash \mathrm{Prov}_{\mathrm{PA}}(\ulcorner \bot \urcorner) \to H, \qquad \mathrm{PA} \vdash N \to \mathrm{Con}(\mathrm{PA}).$$

The compiled word, scheduler, and trace construction are checked separately. The trace endpoint is `RefutationHalting.lean` and the final consistency implication is in `FinalEndpoint.lean`.

7 Two-sided independence

Proof of Theorem 1.1. Suppose first that PA is consistent. The external search soundness argument gives literal blank-tape nonhalting and hence $\mathbb{N} \models N$. If PA proved N, the internal implication of Section 6 would give a PA proof of its own syntactic consistency. Gödel's second incompleteness theorem [7] forbids this.

The standard natural-number structure satisfies the defining equations of arithmetic and every induction instance. Soundness of first-order derivations therefore yields external consistency and truth in $\mathbb{N}$ of every PA theorem. In particular a PA proof of H would give $\mathbb{N} \models H$, contradicting $\mathbb{N} \models N$. Thus $\mathrm{PA} \nvdash H$. Since classical PA proves $\neg N \leftrightarrow H$, it also cannot prove $\neg N$. The standard model discharges the consistency premise used in the first part. □

The Lean endpoint is `TwoSidedIndependence.lean`. `Solution.lean` transfers the exact formula and provability predicate to the statements in `Challenge.lean`. The same Solution identifies its tape semantics with the literal machine and transfers the blank-tape nonhalting proof. The two statements concern the particular machine defined by the table in Section 2.

References

[1] T. Radó. On non-computable functions. *Bell System Technical Journal* 41 (1962), 877–884. https://doi.org/10.1002/j.1538-7305.1962.tb00480.x.

[2] A. Yedidia and S. Aaronson. A relatively small Turing machine whose behavior is independent of set theory. *Complex Systems* 25 (2016), 297–327. https://arxiv.org/abs/1605.04343.

[3] J. Chang. Nonhalting and PA-independence for a 211-state Turing machine. https://github.com/jamesyc/BusyBeaver/tree/1654314dd6892371ea38e488a7aa501d2aa55a0c.

[4] E. Glazer. *K230 Rocq verification*. https://github.com/elliotglazer/k230-rocq/tree/3142e4d.

[5] *PA238 Rocq development*. https://github.com/jeff87654/busy-beaver/tree/84049b5/PA/m238.

[6] V. Reshetnikov. *ProveIt*, DominatedCore.v. https://github.com/VladimirReshetnikov/ProveIt/tree/6809609.

[7] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik* 38 (1931), 173–198. https://doi.org/10.1007/BF01700692.

[8] FormalizedFormalLogic. *Foundation: Formalized mathematical logic in Lean 4*. https://github.com/FormalizedFormalLogic/Foundation.